



■ PrixMaroc

Guide complet de développement

Application de comparaison de prix pour ménages marocains

OCR Tickets

IA Recommendations

Scraping GMS

Maps & Itinéraires

6 Phases

20 Semaines

30+ Prompts

3 Outils Claude

Par où commencer ?

Commence toujours par l'étape 1.1 — le schéma de base de données. Les données initiales s'insèrent dans des tables qui n'existent pas encore. Voici l'ordre exact à respecter :

#	Étape	Outil	Durée
1	Schéma base de données (SQL complet)	Claude Chat	2-3h
2	Architecture technique	Claude Chat	1-2h
3	Setup projet + Docker	Claude Code	2-3h
4	Services OCR + Scraper	Claude Code	1 semaine
5	Script seed (données initiales)	Claude Code	1 jour
6	API produits, prix, magasins	Claude Code	1 semaine
7	Service IA + recommandations	Claude Code	1 semaine
8	Frontend mobile (4 écrans)	CoWork + Code	2 semaines
9	Panel admin	CoWork	1 semaine
10	Notifications + Maps + Deploy	Claude Code	1 semaine

■ ■ Règle d'or Ne jamais passer à l'étape suivante si la précédente n'est pas fonctionnelle. Tester chaque service avec au moins 3 cas réels avant de continuer.

Table des matières

Phase 1

Conception & Architecture

- Étape 1.1 — Schéma base de données
- Étape 1.2 — Architecture technique

Phase 2

Backend & Base de données

- Étape 2.1 — Initialisation projet + Docker
- Étape 2.2 — Service OCR (lecture tickets)
- Étape 2.3 — Service de scraping GMS
- Étape 2.4 — API produits, prix & magasins

Phase 3

Intelligence Artificielle

- Étape 3.1 — Moteur de recommandations
- Étape 3.2 — Dashboard & KPIs

Phase 4

Frontend Mobile

- Étape 4.1 — Setup React Native
- Étape 4.2 — Écran Scanner OCR
- Étape 4.3 — Écran Comparaison des prix
- Étape 4.4 — Écran Liste d'achats

Phase 5

Profil & Administration

- Étape 5.1 — Profil utilisateur modifiable
- Étape 5.2 — Panel administrateur

Phase 6

Intégrations & Déploiement

- Étape 6.1 — Notifications push
- Étape 6.2 — Géolocalisation & itinéraires
- Étape 6.3 — Déploiement production
- Bonus — Script seed données initiales

Phase 1

Conception & Architecture

Avant d'écrire la moindre ligne de code, il faut définir la structure complète. Cette phase se fait entièrement dans Claude Chat — pas de code, que de la réflexion.

Étape 1.1 Schéma base de données (SQL complet)

Claude Chat

Définir toutes les tables, colonnes, relations et index. C'est la fondation de toute l'application.

■ Prompt à copier-coller

Tu es architecte logiciel senior. Je veux créer une application mobile appelée "PrixMaroc" pour les ménages marocains permettant de comparer les prix dans les grandes surfaces (Marjane, Carrefour, Label'Vie, BIM, Atacadão).

Crée le schéma de base de données complet PostgreSQL avec toutes les tables pour :

- Produits (champs nutritionnels : calories, protéines, lipides, glucides, nutri-score, photo_url, barcode EAN)
- Prix par magasin avec historique
- Utilisateurs (profil : ville, nombre_personnes, budget_mensuel)
- Tickets de caisse scannés (OCR)
- Listes d'achats (hebdomadaire, bi-mensuelle, mensuelle)
- Magasins avec géolocalisation GPS
- Promotions
- Historique des économies

Donne le SQL complet avec les relations, index et contraintes.

Ajoute des commentaires explicatifs sur les choix de modélisation.

✓ **Résultat attendu :** Fichier schema.sql complet à sauvegarder — sera utilisé à l'étape 2.1

Étape 1.2 Architecture technique

Claude Chat

Valider le stack technique et planifier l'infrastructure avant de coder.

■ Prompt à copier-coller

Pour l'application PrixMaroc, propose une architecture technique complète :

Stack à évaluer :

- Frontend : React Native (iOS + Android) + version PWA
- Backend : FastAPI (Python) – justifie pourquoi vs Node.js pour ce projet
- Base de données : PostgreSQL + Redis (cache prix)
- OCR : Tesseract + modèle fine-tuné pour tickets marocains
- Scraping : Scrapy + Playwright
- Storage : Cloudflare R2 pour les photos produits

- Maps : Google Maps API

Donne :

1. Diagramme d'architecture en ASCII
2. Structure des dossiers du projet
3. Variables d'environnement nécessaires
4. Estimation des coûts d'hébergement pour 10 000 utilisateurs (MAD/mois)

✓ **Résultat attendu :** Document architecture.md — référence pour toute l'équipe

Phase 2

Backend & Base de données

Construction du cœur technique : base de données, OCR, scraping et API REST. Tout se passe dans Claude Code — copier-coller les prompts directement.

Étape 2.1 Initialisation projet + Docker

[Claude Code](#)

Créer la structure complète du projet et faire tourner l'environnement local.

■ Prompt à copier-coller

```
Initialise un projet FastAPI pour PrixMaroc avec :

1. Structure complète du projet :
/backend
/app
/models (SQLAlchemy)
/routers (produits, prix, utilisateurs, magasins, listes)
/services (ocr, scraper, ia, maps)
/schemas (Pydantic)
/utils
/migrations (Alembic)
/tests
docker-compose.yml avec PostgreSQL + Redis + backend

2. Crée tous les modèles SQLAlchemy depuis ce schéma SQL : [COLLER schema.sql]

3. Configure Alembic pour les migrations

4. Crée le fichier docker-compose.yml complet

Assure-toi que tout démarre avec : docker-compose up
```

✓ **Résultat attendu :** Projet qui tourne localement sur `http://localhost:8000`

Étape 2.2 Service OCR — lecture tickets

[Claude Code](#)

Parser automatiquement les tickets de caisse marocains en données structurées.

■ Prompt à copier-coller

```
Crée un service OCR complet pour lire les tickets de caisse marocains.

Fichier : /app/services/ocr_service.py

Le service doit :
```

1. Accepter une image (JPEG/PNG/PDF) d'un ticket de caisse
2. Utiliser Tesseract avec langue française + arabe (ara+fra)
3. Parser le texte pour identifier :
 - Nom du magasin (Marjane, Carrefour, Label'Vie, BIM, Atacadão)
 - Date et heure de l'achat
 - Liste des produits : nom, quantité, prix unitaire, prix total
 - Total du ticket
4. Normaliser les noms de produits (match avec BDD)
5. Retourner un JSON structuré

Preprocessing OpenCV : débruitage, redressement, contraste.

Inclure les tests unitaires avec tickets marocains simulés.

✓ **Résultat attendu :** Endpoint POST /api/ocr/scan fonctionnel

Étape 2.3 Service de scraping GMS

[Claude Code](#)

Récupérer automatiquement les catalogues et prix des grandes surfaces.

■ Prompt à copier-coller

Crée un service de scraping éthique pour les catalogues GMS marocains.

Fichier : /app/services/scrapper_service.py

À implémenter :

1. Scraper pour Marjane.ma : nom produit, prix, catégorie, photo
2. Scraper pour Carrefour Maroc
3. Scraper pour Label'Vie
4. Scraper générique (configurable via admin)

Features :

- Respect du robots.txt
- Rate limiting (max 1 requête/2s par site)
- Rotation User-Agent
- Gestion CAPTCHAs (pause + alerte admin)
- Déduplication (fuzzy matching sur le nom)
- Stockage photos sur Cloudflare R2
- Scheduling APScheduler (quotidien 3h du matin)
- Logs pour l'admin

Interface admin FastAPI :

- Déclencher scraping manuel
- Voir statut des scrapers
- Ajouter/modifier URLs cibles

✓ **Résultat attendu :** Scrapers qui tournent en background, prix en BDD

Étape 2.4 API produits, prix & magasins

[Claude Code](#)

Exposer toutes les données via une API REST robuste avec cache Redis.

■ Prompt à copier-coller

Crée les routers FastAPI complets pour PrixMaroc :

1. `/api/products` :
 - GET `/products?search=&category=&city=`
 - GET `/products/{id}` (détail + prix par magasin)
 - GET `/products/{id}/price-history` (30/90 jours)
 - GET `/products/compare?ids=1,2,3`

2. `/api/prices` :
 - GET `/prices/cheapest?product_id=&city=`
 - GET `/prices/promotions?city=&user_id=`

3. `/api/stores` :
 - GET `/stores/nearby?lat=&lng=&radius=`
 - GET `/stores/{id}/products`
 - GET `/stores/route-optimize?store_ids=`

Pour chaque endpoint :

- Validation Pydantic complète
- Cache Redis (TTL 1h prix, 24h produits)
- Pagination
- Gestion d'erreurs HTTP appropriée

✓ **Résultat attendu :** API v1 documentée sur `/docs` (Swagger auto-généré)

Phase 3

Intelligence Artificielle

Intégration de l'IA pour personnaliser l'expérience : recommandations, listes automatiques et alertes intelligentes basées sur les habitudes d'achat.

Étape 3.1 Moteur de recommandations IA

[Claude Code](#)

Analyser les habitudes d'achat pour générer des listes et alertes personnalisées.

■ Prompt à copier-coller

Crée le service IA de recommandations pour PrixMaroc.

Fichier : /app/services/ia_service.py

Ce service doit :

1. ANALYSE DES HABITUDES D'ACHAT :

- Depuis l'historique des tickets scannés de l'utilisateur
- Calculer : fréquence d'achat par produit, budget moyen par catégorie, sensibilité aux promos
- Détecter les produits récurrents (achetés > 80% des semaines)

2. GÉNÉRATION DE LISTE D'ACHATS :

Endpoint : POST /api/ai/generate-list

Input : user_id, type (hebdo|bi-mensuel|mensuel), budget_max

Output : liste avec produits, quantités, prix estimés, magasin conseillé

3. ALERTES PROMOTIONS PERSONNALISÉES :

- Score pertinence = fréquence_achat × (1 - prix_promo/prix_normal)
- Notifier si score > seuil configurable

Utilise Claude API (claude-sonnet) pour le raisonnement.

Donne le prompt système complet utilisé pour l'appel à Claude.

✓ Résultat attendu : Endpoint /api/ai/generate-list fonctionnel

Étape 3.2 Dashboard & KPIs

[Claude Code](#)

Agréger toutes les métriques utilisateur en une seule requête optimisée.

■ Prompt à copier-coller

Crée l'endpoint dashboard pour PrixMaroc.

GET /api/dashboard/{user_id}

Doit retourner en une requête :

```
{
  "economies": {
    "ce_mois": 234.50,
    "cumul_annee": 1820.00,
    "graphique_30j": [...]
  },
  "statistiques": {
    "tickets_scannes": 47,
    "produits_suivis": 128,
    "magasin_prefere": "Marjane Hay Riad",
    "score_econome": 78
  },
  "alertes_actives": [...],
  "prochaine_liste": {...},
  "magasin_conseille": {
    "nom": "Carrefour Anfa",
    "distance": "2.3 km",
    "economie_estimee": 45.00
  }
}
```

Optimise avec requêtes SQL agrégées + cache Redis.

Inclure les migrations Alembic nécessaires.

✓ **Résultat attendu :** Dashboard temps réel avec toutes les métriques

Phase 4

Frontend Mobile

Développement de l'application mobile React Native avec 4 écrans principaux. Utiliser CoWork pour les écrans visuels complexes et Claude Code pour le setup.

Étape 4.1 Setup React Native

Claude Code

Initialiser le projet mobile avec toutes les dépendances et navigation.

Prompt à copier-coller

Initialise le projet React Native pour PrixMaroc.

Setup complet avec :

- Expo SDK (dernière version stable)
- React Navigation v6 (tabs + stack)
- Zustand pour le state management
- React Query pour les appels API + cache
- NativeWind (Tailwind pour RN)
- Expo Camera pour le scan OCR
- Expo Location pour la géolocalisation
- react-native-maps

Navigation :

- Tab 1 : Accueil / Dashboard
- Tab 2 : Scanner (OCR ticket)
- Tab 3 : Comparer les prix
- Tab 4 : Mes listes
- Tab 5 : Mon profil

Crée :

1. App.tsx avec navigation complète
2. Palette de couleurs (vert Maroc + moderne)
3. Service API (axios + interceptors + JWT)
4. Types TypeScript pour tous les modèles
5. Composant ProduitCard réutilisable

✓ Résultat attendu : App qui tourne sur simulateur iOS/Android

Étape 4.2 Écran Scanner OCR

CoWork

Interface de scan des tickets avec feedback visuel et correction manuelle.

Prompt à copier-coller

Crée l'écran de scan complet pour PrixMaroc.

Fichier : /screens/ScannerScreen.tsx

L'écran doit :

1. Ouvrir la caméra (Expo Camera) avec cadre de guidage
2. Bouton "Photographier" → envoyer à /api/ocr/scan
3. Animation de lecture pendant le traitement
4. Afficher les produits détectés avec cases à cocher
5. Possibilité de corriger un nom manuellement
6. Bouton "Valider et ajouter à mon historique"
7. Si produit non reconnu : "Rechercher manuellement"
8. Résumé : total ticket, économies potentielles

UX : feedback visuel à chaque étape, messages bienveillants en français avec option arabe.

✓ **Résultat attendu :** Écran scanner complet et testé sur vrai téléphone

Étape 4.3 Écran Comparaison des prix

CoWork

Moteur de recherche avec tableau comparatif des prix par magasin.

■ Prompt à copier-coller

Crée l'écran de comparaison des prix pour PrixMaroc.

Fichier : /screens/ComparaisonScreen.tsx

Fonctionnalités :

1. Barre de recherche avec autocomplétion API
2. Cards produits avec :
 - Photo réelle du produit
 - Nom + marque + contenance
 - Tableau prix par magasin (trié moins cher → plus cher)
 - Badge vert "MEILLEUR PRIX"
 - Infos nutritionnelles expansibles (cal/100g, protéines, lipides)
 - Bouton "Ajouter à ma liste"
 - Graphique mini prix sur 30 jours
3. Filtres : catégorie, marque, magasin, budget max
4. Tri : prix croissant, économie max, proximité
5. Comparateur multi-produits (jusqu'à 4)

Support RTL pour l'arabe. Fort contraste (lisible au soleil).

✓ **Résultat attendu :** Comparateur avec données réelles de la BDD

Étape 4.4 Écran Liste d'achats

[Claude Code](#)

Gestionnaire de listes hebdo/mensuelle avec génération IA et mode courses.

■ Prompt à copier-coller

Crée l'écran de gestion des listes d'achats PrixMaroc.

Fichier : /screens/ListeAchatsScreen.tsx

Features :

1. 3 onglets : Hebdomadaire / Bi-mensuelle / Mensuelle

2. Bouton "Générer avec l'IA" → /api/ai/generate-list

Animation : "L'IA analyse vos habitudes..."

3. Liste générée groupée par catégorie avec :

- Prix estimé total MAD
- Option split entre 2 magasins pour max économies
- Bouton "Optimiser le trajet" → Maps

4. Édition manuelle : ajouter, supprimer, changer quantité

5. Partage WhatsApp

6. Mode "En courses" : cocher items, total restant affiché

7. Sauvegarde locale AsyncStorage (offline)

Animations Reanimated pour les coches et transitions.

✓ **Résultat attendu :** Listes fonctionnelles avec génération IA

Phase 5

Profil & Administration

Gestion du profil utilisateur et panel administrateur pour superviser les scrapers, valider les données et gérer les produits.

Étape 5.1 Profil utilisateur modifiable

[CoWork](#)

Toutes les informations personnalisables du profil utilisateur.

■ Prompt à copier-coller

Crée l'écran Profil utilisateur complet pour PrixMaroc.

Fichier : /screens/ProfilScreen.tsx

Sections modifiables :

1. INFORMATIONS PERSONNELLES :

Prénom, email, téléphone, photo de profil

2. MON FOYER :

- Nombre de personnes (slider 1-10)
- Tranches d'âge (adultes, enfants, seniors)

3. MON BUDGET :

- Budget mensuel courses (MAD)
- Budget par catégorie (optionnel)
- Objectif d'économies mensuel

4. MA VILLE / ZONE :

- Ville (liste : Casablanca, Rabat, Marrakech, Fès, etc.)
- Rayon max déplacement (km)
- Magasins préférés (toggle)

5. PRÉFÉRENCES :

- Langue (Français / Arabe / Darija)
- Notifications par type
- Régime alimentaire (végétarien, halal strict, sans gluten)

Validation temps réel, sauvegarde auto, toast confirmation.

✓ **Résultat attendu :** Profil complet avec toutes les préférences sauvegardées

Étape 5.2 Panel administrateur

[Claude Code](#)

Interface web pour gérer les scrapers, produits et superviser l'application.

■ Prompt à copier-coller

Crée le panel admin web pour PrixMaroc.

Tech : React + Vite + Tailwind (webapp séparée)

Route protégée : /admin (JWT rôle admin)

Pages :

1. /admin/dashboard : stats temps réel (users actifs, tickets scannés, prix mis à jour, erreurs scraping)

2. /admin/scrapers :

- Liste scrapers (nom, URL, fréquence, dernier run, statut)
- Bouton "Lancer maintenant"
- Formulaire ajout nouveau scraper (URL, sélecteurs CSS)
- Logs temps réel (WebSocket)
- Alertes CAPTCHAS

3. /admin/products :

- Table avec filtres et recherche
- Édition inline : photo, nutritionnel, catégorie
- Import CSV en masse
- Gestion doublons (fusionner similaires)
- Auto-complétion via Open Food Facts API

4. /admin/prices :

- Historique par produit/magasin
- Détection anomalies (prix x3 en 1 jour)

Utilise React Query + data-grid pour les performances.

✓ **Résultat attendu :** Panel admin accessible sur /admin avec login sécurisé

Phase 6

Intégrations & Déploiement

Notifications push, géolocalisation, optimisation d'itinéraires et mise en production. Cette phase finalise l'application et la rend disponible aux utilisateurs.

Étape 6.1 Notifications push

[Claude Code](#)

Alertes personnalisées : promos, rappels de listes, récap économies.

■ Prompt à copier-coller

Implémente le système de notifications push pour PrixMaroc.

Backend (FastAPI) :

1. Intégrer Firebase Cloud Messaging (FCM)
2. /app/services/notification_service.py
3. Types de notifications :
 - PROMO_ALERTE : "Le lait Centrale que vous achetez est à -30% chez Marjane"
 - LISTE_RAPPEL : "N'oubliez pas votre liste (budget: 350 MAD)"
 - ECONOMIE_HEBDO : "Cette semaine vous avez économisé 67 MAD"
 - PRIX_BAISSE : "Huile Olor 5L → 89 MAD chez Carrefour"

Scheduler APScheduler :

- Dimanche 9h : notification liste hebdo
- Chaque jour 8h : scan promos pertinentes
- Lundi matin : rapport économies semaine

Frontend : Expo Notifications + deep links

Règle : max 2 notifications/jour/utilisateur.

✓ **Résultat attendu :** Notifications qui arrivent sur le téléphone

Étape 6.2 Géolocalisation & itinéraires

[Claude Code](#)

Trouver les magasins les moins chers les plus proches et optimiser le trajet.

■ Prompt à copier-coller

Implémente la géolocalisation et itinéraires pour PrixMaroc.

1. Backend - /api/stores/route-optimize :

Input : user_location (lat/lng), liste produits avec store_ids

Logique :

- Itinéraire optimal pour 1 à 3 magasins
- Pondérer économies vs distance
- Règle : 2ème magasin seulement si économie > coût trajet

(0.8 MAD/km estimé en ville marocaine)

Output : magasins ordonnés, économie totale, URL Google Maps

2. Frontend - MagasinsProchesScreen.tsx :

- Carte react-native-maps avec markers colorés

(vert = moins cher, orange = intermédiaire)

- Card résumé : "Marjane + BIM : économie 78 MAD, 4.2 km"

- Bouton "Lancer l'itinéraire" → Google Maps / Waze

3. Seed magasins : SQL avec tous les Marjane, Carrefour, Label'Vie, BIM, Atacado du Maroc avec coordonnées GPS et horaires.

✓ **Résultat attendu :** Carte interactive avec itinéraire optimal

Étape 6.3 Déploiement production

[Claude Code](#)

Mettre l'application en ligne avec CI/CD, SSL et monitoring.

■ Prompt à copier-coller

Crée la configuration de déploiement complète pour PrixMaroc.

Infrastructure (budget Maroc startup) :

- VPS OVH/DigitalOcean (4 vCPU, 8GB RAM) ~400 MAD/mois
- PostgreSQL : Supabase free tier
- CDN/Storage : Cloudflare R2 (gratuit jusqu'à 10GB)

Créer :

1. Dockerfile optimisé FastAPI (multi-stage build)
2. docker-compose.prod.yml (backend + nginx + certbot SSL)
3. nginx.conf pour reverse proxy SSL
4. Script deploy.sh : pull, build, migrate, restart sans downtime
5. GitHub Actions CI/CD :
 - Tests auto sur PR
 - Deploy auto sur push main
6. Backup PostgreSQL quotidien vers Cloudflare R2
7. Monitoring UptimeRobot
8. .env.production documenté

README.md déploiement en français.

✓ **Résultat attendu :** App accessible sur <https://prixmaroc.ma>

Bonus — Script seed : données initiales

■ Quand ?

Ce script se lance APRÈS l'étape 2.1 (BDD créée et migrations jouées). Sans tables existantes, le seed échoue. Ordre impératif : structure d'abord, données ensuite.

Ce script importe ~5 000 produits marocains réels depuis Open Food Facts et remplit la BDD avec les magasins géolocalisés.

■ Prompt seed_database.py

Crée un script Python seed_database.py pour PrixMaroc qui :

1. Appelle l'API Open Food Facts pour récupérer 5000 produits vendus au Maroc (paramètre : country=morocco, champs : nom, barcode, calories, protéines, lipides, glucides, nutri-score, photo_url)

2. Normalise les données :

- Calories, protéines, lipides, glucides par 100g
- Noms en français priorité
- Catégories mappées vers nos catégories app

3. Télécharge et stocke les photos sur Cloudflare R2

4. Insère en base PostgreSQL via SQLAlchemy

- Gère les doublons par barcode EAN
- Skip les produits sans nom ou prix

5. Seed magasins : insère tous les Marjane, Carrefour, Label'Vie, BIM, Atacadão du Maroc avec coordonnées GPS, horaires et ville

6. Log final : X produits importés, Y photos, Z magasins, W erreurs

Durée estimée : 30-60 minutes pour 5000 produits.

Conseils par outil Claude

Outil	Idéal pour	Éviter
Claude Chat	Architecture, schémas, choix techniques, debugging conceptuel	Génération de fichiers longs
Claude Code	Backend complet, scripts, API, tests, déploiement	Interfaces visuelles complexes
CoWork	Écrans mobiles, composants UI, formulaires, multi-fichiers	Scripts backend purs

Checklist avant le lancement

Backend

- API /docs accessible et tous les endpoints testés
- OCR reconnaît au moins 5 vrais tickets marocains
- Scraper tourne sans erreur sur Marjane et Carrefour
- Cache Redis fonctionnel (temps de réponse < 200ms)
- BDD seedée avec 5000+ produits et 200+ magasins

Frontend

- Scan OCR < 10 secondes sur iPhone et Android
- Comparaison affiche photos réelles et infos nutritionnelles
- Génération liste IA < 5 secondes
- Mode courses fonctionne hors-ligne
- RTL arabe correct sur tous les écrans

Production

- SSL HTTPS configuré (Let's Encrypt)
- Backup BDD quotidien testé et restauré
- Notifications push reçues sur vrais téléphones
- UptimeRobot alerte en cas de panne
- Monitoring erreurs (Sentry ou logs)

Bonne chance avec PrixMaroc !

Ce guide contient tous les prompts nécessaires pour construire l'application de A à Z. Commence par l'étape 1.1 dans Claude Chat, et avance méthodiquement en validant chaque étape avant de passer à la suivante.

Stack technique	Durée estimée	Budget infra
FastAPI + React Native PostgreSQL + Redis Expo + NativeWind	20 semaines 6 phases 30+ prompts prêts	~400 MAD/mois VPS + BDD managée Cloudflare R2 gratuit